

17

Cyber Security
Codec Technologies India
Internship 2026

Digital Forensics Toolkit

Triage: hash, carve, timeline, memory, custody

AT A GLANCE

13

Files hashed

2 / 2

JPEGs carved

39

Timeline events

5

Memory alerts

6 (OK)

Custody entries

12 / 12

Tests re-run

synthetic

Evidence

VERDICT

TESTS 12/12

RE-RUN HERE

DECLARED SYNTHETIC

DEFENSIVE ONLY

EXISTING PDF KEPT

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Objective & brief mapping

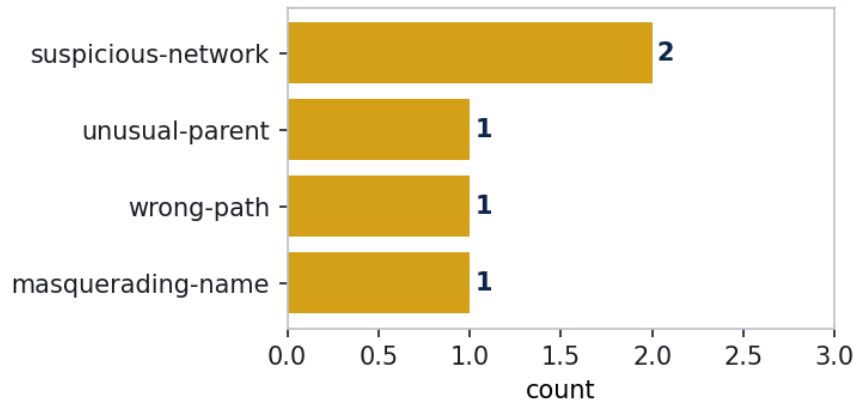
The brief asked for a digital forensics toolkit. I built the first passes an analyst actually runs on seized evidence: hash everything for chain-of-custody, identify file types by magic bytes, pull metadata, extract strings, carve deleted files from a raw image, build a MAC-time timeline, and triage memory to reconstruct a process tree and flag suspicious processes — every step logged into an append-only, hash-chained custody record.

Architecture

main.py runs eight modules over evidence/: hashing (MD5+SHA-256 manifest), filetype (magic-byte ID + mismatch flags), metadata (EXIF, PDF), strings, carver (JPEG SOI/EOI recovery with no filesystem metadata), timeline, memory (pstree + alerts), and custody (SHA-256 hash-chained log).

```
evidence/ (synthetic) -> main.py -> forensics/
hashing | filetype | metadata | strings | carver
timeline | memory (pstree+alerts) | custody (hash-chain)
-> forensic_report.md
```

Memory-triage alerts by category (5 alerts / 13 processes)



Memory-triage alerts by category, from memory_analysis.json.

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python main.py # generate evidence if missing, full triage (<1 s)
python3 unittest-style runner over tests/ # 12 tests
```

The artefacts each figure and table above are drawn from:

- **memory_analysis.json** — pstree + 5 alerts
- **custody_log.json** — hash-chained log
- **custody_manifest.csv / timeline.csv** — hashes + MAC timeline
- **carved/** — recovered JPEGs
- **forensic_report.md** — case report

Method & implementation

Carving scans disk.img for JPEG markers and validates each recovery with Pillow. Memory heuristics flag masquerading names, wrong-path system binaries, unusual parents and C2-style beacons — exactly the checks you'd apply to Volatility 3 pslist/pstree/netscan output. Every action appends to the custody chain; verify() detects any tampering.

Results

From results/memory_analysis.json and custody_log.json. Benign processes (chrome.exe, legit svchost.exe) are correctly not flagged.

PID / metric	Finding	Category
3050 svchost.exe	typosquat of svchost	masquerading-name
3120 lsass.exe	runs from AppData	wrong-path
3050	beacon 203.0.113.66:4444	suspicious-network
3300 powershell.exe	beacon 198.51.100.9:8080	suspicious-network
Carving / timeline	2/2 JPEGs, 39 events ordered	OK
Custody	6 entries, integrity	OK

Triage passes

Pass	Output	Real tool
hash + custody	manifest, hash-chain	md5sum + write-blocker
carve	recovered JPEGs	PhotoRec
timeline	MAC events	Plaso

Pass	Output	Real tool
memory	pstree + alerts	Volatility 3

Verification

Re-ran here, 12/12 (~0.13 s). All analysis logic is pure-Python over bundled synthetic evidence.

Test group	Checks	Result
Hashing	stable, complete, tampering detected	PASS
Carving/timeline	both JPEGs valid, chronological	PASS
Memory	typosquat, wrong-path, C2 flagged	PASS
Custody	immutable, append-only, tamper detected	PASS
Total	12 tests	12/12

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

All analysis logic is real; the memory artifact set stands in for a RAM image I cannot lawfully ship.

Ran here	Real tool	Command
memory heuristics	Volatility 3	vol -f mem.raw windows.pstree
JPEG marker carving	PhotoRec	carve fragmented / other formats
MAC timeline	Plaso / log2timeline	parse \$MFT / inode structures

Ethics, data integrity & declared substitutions

Educational and defensive. **Data integrity:** every piece of evidence is synthetic and generated locally — the disk image (with two genuine embedded JPEGs), EXIF photos, PDF and memory artifact set (three planted threats); names are made-up and IPs are reserved documentation ranges. **Declared substitution:** against a real RAM image you'd use Volatility 3 (windows.pslist/pstree/netscan/malfind); memory_artifacts.json is a declared-synthetic stand-in shaped exactly like that output, and my heuristics run over it. **Existing deliverable kept:** the project's results/evidence PDF was left untouched; this is the standard docs/17_Report.pdf added alongside.

Learnings

The hash-chained custody log was the detail that made this feel like real forensics: integrity you can prove, not just claim. Writing the memory heuristics as the same checks an analyst applies to Volatility output meant the synthetic stand-in taught the real workflow, not a toy version of it.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- Carving is marker-based (JPEG SOI/EOI); no fragmented-file handling.
- The file-type table is a small transparent subset, not full libmagic.
- Timeline uses host MAC times (POSIX ctime is metadata-change, not birth time).
- Memory analysis is a synthetic stand-in; see the Volatility workflow.
- 13 files hashed; 2/2 JPEGs carved; 39 timeline events; 5 memory alerts.
- Every action appends to a SHA-256 hash-chained custody log; verify() detects tampering.
- All evidence is synthetic and generated locally; IPs are documentation ranges.
- The project's own evidence PDF was kept untouched; this report added alongside.
- 13 files hashed with MD5 + SHA-256; re-hash verification intact.
- 2/2 embedded JPEGs carved from disk.img and validated by Pillow.
- 39 MAC-time events, chronologically ordered.
- Memory triage: 13 processes, 5 alerts.
- Alerts: svch0st typosquat, wrong-path lsass, two C2 beacons.
- Chain of custody: 6 entries, integrity OK.
- Benign chrome.exe and legit svchost.exe not flagged.

17

Cyber Security
Codec Technologies India
Internship 2026

Digital Forensics Toolkit

Triage: hash, carve, timeline, memory, custody

AT A GLANCE

13

Files hashed

2 / 2

JPEGs carved

39

Timeline events

5

Memory alerts

6 (OK)

Custody entries

12 / 12

Tests re-run

synthetic

Evidence

VERDICT

TESTS 12/12

RE-RUN HERE

DECLARED SYNTHETIC

DEFENSIVE ONLY

EXISTING PDF KEPT

Eswar Mahalingam

Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam